

BINARY SEARCH

Why does it take only $\log(n)$ not $O(n) + \log(n)$?

```
# Binary Search  --> Log(n)
# Log(n)
# l=0 h=len(arr)-1
```

High = len(arr) or sum(arr) → in this case it takes $O(n)$ to calculate len(arr) or sum(arr)

1. Find First and Last Position of Element in Sorted Array

<https://leetcode.com/problems/find-first-and-last-position-of-element-in-sorted-array/>

```
# function to find the first occurrence of the target in the sorted array
def firstOcc(arr, target):
    low, high = 0, len(nums) - 1
    index = -1
    while low <= high:
        m = (low + high) // 2
        if nums[m] == target:
            # If the current element is equal to the target, update the index and search in the left half
            index = m
            high = m - 1 ### core logic
        elif target > nums[m]:
            # If the target is greater, search in the right half
            low = m + 1
        else:
            # If the target is smaller, search in the left half
            high = m - 1
    return index
```

```
# Helper function to find the last occurrence of the target in the sorted array
def lastOcc(arr, target):
    low, high = 0, len(nums) - 1
    index = -1
    while low <= high:
        m = (low + high) // 2
        if nums[m] == target:
            # If the current element is equal to the target, update the index and search in the right half
            index = m
            low = m + 1 ### core logic
        elif target > nums[m]:
            # If the target is greater, search in the right half
            low = m + 1
        else:
            # If the target is smaller, search in the left half
            high = m - 1
    return index

# Find the first and last occurrences of the target using helper functions
first = firstOcc(nums, target)
last = lastOcc(nums, target)
# Return a list containing the first and last occurrences of the target
return [first, last]
```

2. Search in Rotated Sorted Array

<https://leetcode.com/problems/search-in-rotated-sorted-array/description/>

```
i=nums.index(min(nums)) # index of the minimum ele in nums
ans=-1
#1st bs
low , high = 0, i-1 # Set the range for the first binary search
                    # on the left half of the minimum element
while low<=high:
    mid=(low+high)//2
    if nums[mid] == target:
        ans=mid
    if target<nums[mid]:
        high =mid-1
    else:
        low=mid+1
```

```
#2st bs
low , high = i, len(nums)-1 # Set the range for the second binary search
                             # on the right half of the minimum element

while low<=high:
    mid=(low+high)//2
    if nums[mid] == target:
        ans= mid
    if target<nums[mid]:
        high =mid-1
    else:
        low=mid+1

return ans
```

3.Search Insert Position

<https://leetcode.com/problems/search-insert-position/description/>

```
low , high = 0 , len(nums)-1
# Check if the target is greater than the last element
if target > nums[-1]:
    # If so, the target should be inserted at the end return the
position after the last element
    return high+1

while low<=high:

    mid = (low+high)//2

    if nums[mid] == target:
        return mid
    # If the target is smaller, adjust the search range to the left
half
    elif target < nums[mid]:
        high = mid-1
    # If the target is larger, adjust the search range to the right
half
    else:
        low = mid+1

    # If the target is not found, return the position where it should be
inserted
return low
```

4. Search a 2D Matrix

<https://leetcode.com/problems/search-a-2d-matrix/description/>

#1st bs - to find the row in which the search ele is present

#2nd bs - to find search ele in that row

1 st bs

```
#1st bs - to find the row in which the search ele is present

row = len(matrix) # no of rows in the matrix
col = len(matrix[0]) # no of columns in the matrix

low, high = 0, row - 1
ans = -1
# 1st bs
while low <= high:
    mid = (low + high) // 2
    # Check if the last element of the current row is equal to the target
    if matrix[mid][-1] == target:
        # If so, the target is found in the matrix, return True
        return True

    elif matrix[mid][-1] < target:
        ans = mid # possible closest left val
```

```

        low = mid + 1    # Adjust the search range to the right
    else:
        high = mid - 1 # Adjust the search range to the left

# Check if the potential row where the target might be found is the last row
if (ans + 1) == row:
    # If so, the target is not present in the matrix, return False
    return False

```

2nd bs

```

#2nd bs - to find search ele in that row

# Get the array from the potential row where the target might be found
arr = matrix[ans + 1]
# Initialize variables for binary search on the columns
low, high = 0, col - 1

# Perform binary search on the selected row
while low <= high:
    mid = (low + high) // 2
    # Check if the current element in the row is equal to the target
    if arr[mid] == target:
        # If so, the target is found in the matrix, return True
        return True
    # If the current element in the row is larger than the target
    elif arr[mid] > target:
        # Adjust the search range to the left
        high = mid - 1
    else:
        # Adjust the search range to the right
        low = mid + 1

```

5. Find Minimum in Rotated Sorted Array

<https://leetcode.com/problems/find-minimum-in-rotated-sorted-array/description/>

```

# Set the initial range for binary search on the rotated sorted array
low, high = 0, len(nums) - 1

# Perform binary search to find the minimum element in the rotated sorted array
while low < high:
    mid = (low + high) // 2

```

```

    # Check if the middle element is greater than the last element in the
    current range
    if nums[mid] > nums[high]:
        # If so, the minimum element must be in the right half
        low = mid + 1
    else:
        # If not, the minimum element must be in the left half
        high = mid

# Return the minimum element found in the rotated sorted array
return nums[low]

```

NEW PATTERN

max(nums) , sum(nums) - goes on reducing → optimal version

While loop : # outer loop
 For loop : # inner loop

“ Binary Search + check func “

continuous
Min to Max

--> hints for Binary Search

6.First Bad Version

<https://leetcode.com/problems/first-bad-version/description/>

```

# The isBadVersion API is already defined for you.
# def isBadVersion(version: int) -> bool:

class Solution:
    def firstBadVersion(self, n: int) -> int:
        # for i in range(1, n + 1):
        #     if isBadVersion(i):
        #         return i

        low , high = 0,n

        while(low<=high):

```

```

        mid = (low+high)//2

        if isBadVersion(mid):
            ans = mid
            high = mid-1
        else:
            low = mid+1

    return ans

```

7. Capacity To Ship Packages Within D Days

<https://leetcode.com/problems/capacity-to-ship-packages-within-d-days/>

```

def check(mid):
    # check if mid is possible ans
    ul = mid          # upper limit
    countDays = 1     # starting at day 1
    s = 0
    for i in weights:
        if i > ul:     # if ele itself greater than the ul
            countDays = days+1 # we try to fail the test case
                                --> "countDays <= days"
        if s+i > ul:   # s+i --> weight in a day > upper limit
            s = i
            countDays += 1

```

```

        else:
            s += i
        return countDays <= days

low , high = 1 ,sum(weights)
ans=0
while low<=high:
    mid = (low+high)//2
    if check(mid):          # checks the mid = pos ans
        ans = mid          # if true store the pos ans
        # if mid is true then all the rem in right side must be true so, we
search towards left side
        high = mid-1
    else:
        low = mid+1

```

$[1, 2, 3, 4, 5 | 6, 7, 8, 9, 10]$ 5 days
 $w = 15$

$w =$

1	2	3	4	5	6
1	1	15	15	15	15 ✓
2	1	15	15	15	15 ✓
h = 55	27	27	20	16	14
m = 28	14	21	17	15	15
m = 28	28	21	17	15	15

 $s = \text{sum of array}$
 $\frac{\log(s) * n}{s * n}$

8..Koko Eating Bananas

<https://leetcode.com/problems/koko-eating-bananas/description/>

```

# Define a function 'check_can_eat' that takes a parameter 'k'
def check_can_eat(k):

```



```

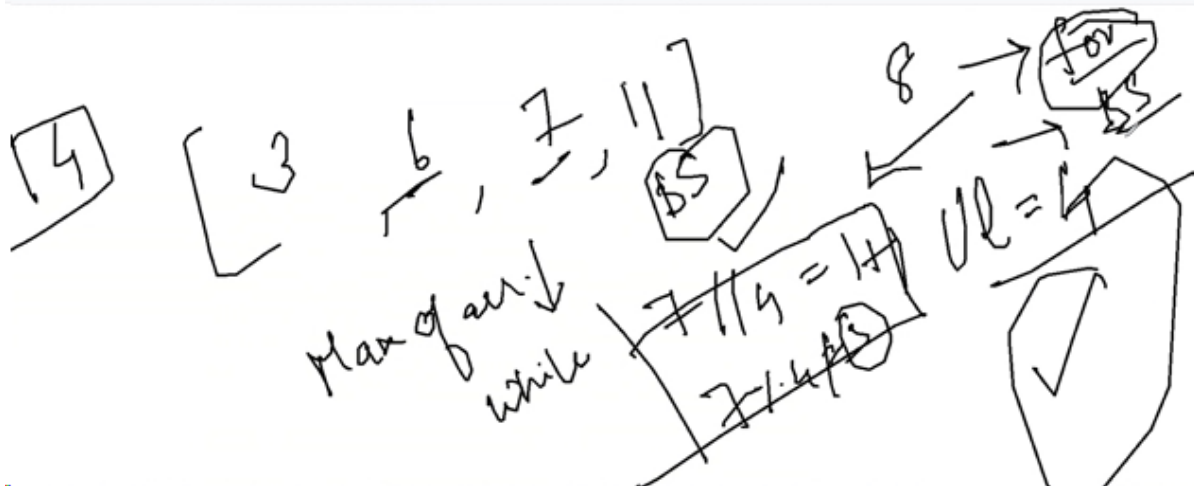
actual_h = 0
for pile in piles:
    # Update 'actual_h' by adding the ceiling division of (pile+k-1) by 'k'
    actual_h += (pile + k - 1) // k
    # Return True if 'actual_h' is less than or equal to the global variable
    # 'h', otherwise return False
    return actual_h <= h

l, r = 0, max(piles)

# Perform a binary search loop while the difference between 'r' and 'l' is
# greater than 1
while r - l > 1:
    # Calculate the midpoint 'm' of the current range
    m = (l + r) // 2
    # If the result of 'check_can_eat(m)' is True, update 'r' to 'm',
    # to search towards let
    if check_can_eat(m):
        r = m
    else:
        l = m

# Return the final result, which is the value of 'r'
return r

```



9. Allocate minimum number of pages

https://www.geeksforgeeks.org/problems/allocate-minimum-number-of-pages0937/1?itm_source=geeksforgeeks&itm_medium=article&itm_campaign=bottom_sticky_on_article

```
#Function to find minimum number of pages.
```

```
def findPages(self,A, N, M):
    #code here
    ans=-1
    if M > N:
        return ans

    def check(m):
        st= 1
        count = 0
        for i in A:
            if count+i >m:
                st+=1
                count = i
            else:
                count += i

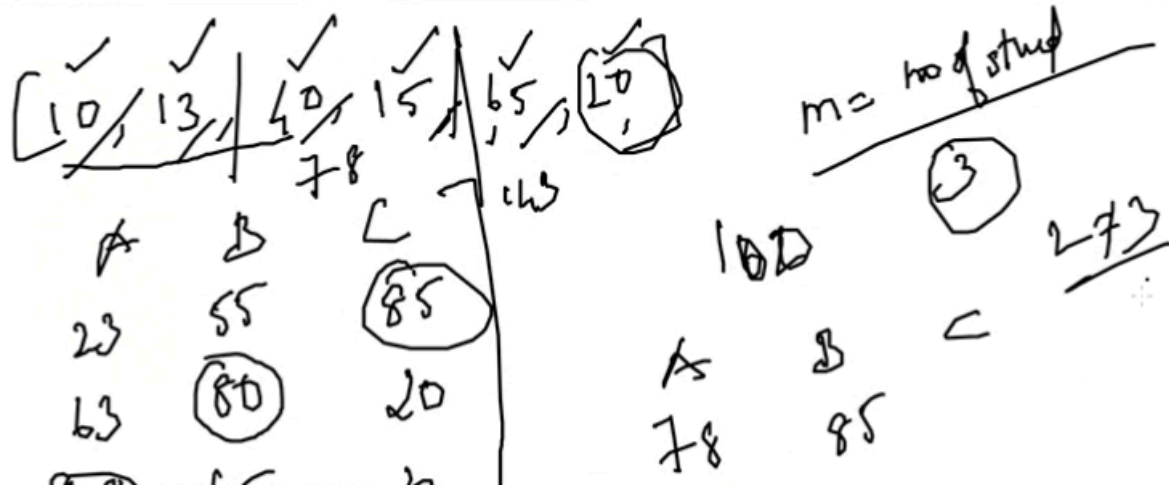
        return st <= M

    low ,high = max(A) , sum(A)
    while low <= high:
        mid = (low+high)//2

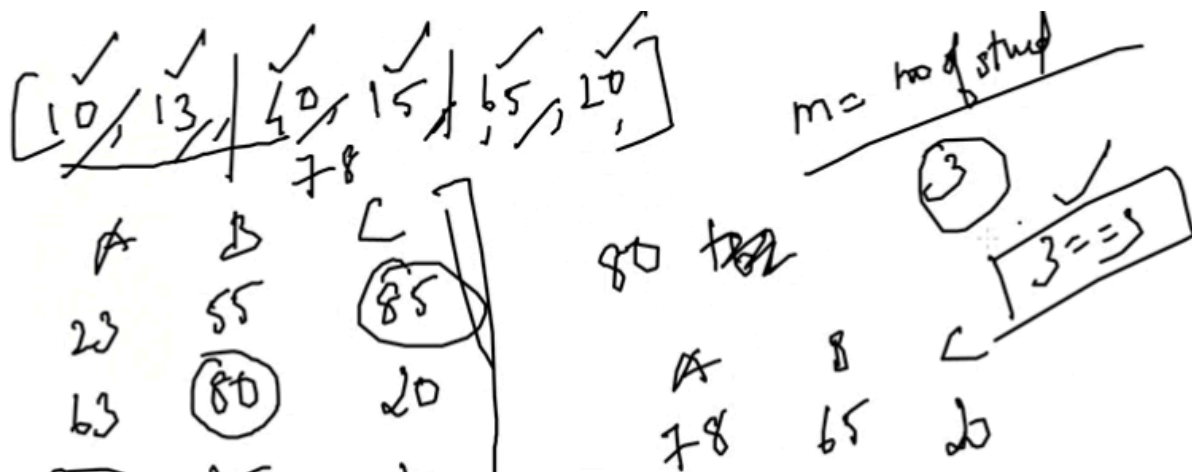
        if check(mid):
            ans = mid
            high = mid-1
        else:
            low = mid+1

    return ans
```

We assume maxi = sum(arr) # sum(arr)=163 and try for even minimum value



We assume with 85 and try for even minimum value



Normal while loop:

For loop:

$st = 1$
 $count = 0$
 for i in arr:
 if $count + i > k$:
 $st++$
 $count = i$
 else:
 $count += i$

$k = 78$
 $count = 0$
 10 23 67 78 65
 $st = 1$ 1 1 1 1 2

10. Split Array Largest Sum

<https://leetcode.com/problems/split-array-largest-sum/description/>

```

def splitArray(self, nums: List[int], k: int) -> int:
    if k==1:
        return (sum(nums))

    def check(mid):
        splits = 1
        count = 0
        for i in nums:
            if count+i > mid: # sum of nums > mid
                count = i    # reinitialize count
                splits += 1   # add a split
            else:
                count += i    # if sumOfNums < mid

        return splits <= k
  
```

```

low , high = max(nums), sum(nums)
ans = -1
while low <= high:
    mid = (low+high)//2
    if check(mid):
        ans = mid
        high = mid-1
    else:
        low = mid+1

return ans

```

same observation as 9 th

Approach : Binary Search

- 1. Base Case:** If k is 1, return the sum of all elements in the array.
- 2. Binary Search:** Use binary search to find the minimum possible maximum sum among k subarrays. Initialize the search range with left as the maximum element and right as the sum of all elements in the array.
- 3. Binary Search Loop:** Continue the loop until left is greater than right. In each iteration, calculate the middle point mid of the current search range.
- 4. Counting Subarrays:** Loop through the array to count how many subarrays can be created with a sum less than or equal to the current mid. If the sum exceeds mid, start a new subarray, and increment the count of splits.
- 5. Adjusting Search Range:** Depending on the count of splits compared to k , adjust the search range. If the count is greater than k , update left to $\text{mid} + 1$. If the count is less than or equal to k , update right to $\text{mid} - 1$.
- 6. Return Result:** Once the binary search converges ($\text{left} > \text{right}$), return the left value as the minimum possible maximum sum.